

## Aufgabe 1

$$\delta = 75 \text{ ms}$$

$$c = \max(|c_{P1}|, |c_{P2}|, |c_{P3}|) = 100 \frac{\mu\text{s}}{\text{s}}$$

$$\Delta t = \frac{\delta}{2c} = 375 \text{ s}$$

Laut Tabelle driften alle Clocks allerdings nur in eine Richtung (dank Vorzeichen). Somit ist die Formel  $\Delta t = \frac{\delta}{2c}$  eigentlich nicht korrekt, da diese miteinberechnet, dass die beiden Clocks „auseinander“ driften. Wenn T aber keinen Drift hat, wird der Synchronisierungsintervall durch

$$\Delta t = \frac{\delta}{c} = 750 \text{ s}$$

gegeben. Wenn allerdings mit „maximale erlaubte Abweichung zwischen den Uhren“ auch die Differenz zwischen  $P_1$ ,  $P_2$  und  $P_3$  gemeint ist, dann ist die maximale Drift-Rate dementsprechend

$$c = \max(|c_{P1} - c_{P2}|, |c_{P1} - c_{P3}|, |c_{P2} - c_{P3}|) = 150 \frac{\mu\text{s}}{\text{s}}$$

$$\Delta t = \frac{\delta}{2c} = 250 \text{ s}$$

Auch hier kann man die Vorzeichen aus der Tabelle so interpretieren, dass stets nur in eine Richtung gedriftet wird. Dann wäre bei der obigen Annahme der Synchronisierungsintervall:

$$\Delta t = \frac{\delta}{c} = 500 \text{ s}$$

Die Fragestellung ist unserer Meinung nach nicht ganz eindeutig.

## Aufgabe 2

TrueTime ist eine von Google entwickelte API, die in Spanner (Googles global verteilte Datenbank) eingesetzt wird. Anders als herkömmliche Zeitquellen liefert TrueTime kein einzelnes Zeitstempel, sondern ein **Intervall [earliest, latest]**, das garantiert die echte aktuelle Zeit enthält.

### Funktionsweise

- Jedes Google-Rechenzentrum hat redundante Zeitquellen: **GPS-Empfänger** und **Atomuhren**
- Ein lokaler *Time Master* aggregiert diese Quellen und erkennt fehlerhafte Quellen durch Mehrheitsentscheid
- Die API gibt `TT.now()` zurück — ein Intervall mit garantierter Fehlergrenze (typisch **1–7 ms**)
- Bevor eine Transaktion committet, wartet Spanner, bis das Intervall vollständig in der Vergangenheit liegt (*commit wait*) — das garantiert externe Konsistenz

## Vorteile gegenüber anderen Methoden

| Methode                  | Problem                                          | TrueTime                                |
|--------------------------|--------------------------------------------------|-----------------------------------------|
| NTP                      | Fehler im Bereich 1–100 ms, keine Fehlergarantie | Bounded uncertainty, typisch < 7 ms     |
| Logische Uhren (Lamport) | Kein Bezug zur Realzeit                          | Echte Wanduhrzeit                       |
| Vektoruhren              | Skalieren schlecht, kein Realzeitbezug           | O(1) pro Knoten                         |
| PTP / IEEE 1588          | Hardware nötig, schwer global skalierbar         | Global, in GCP-Infrastruktur integriert |

**Kernvorteil:** TrueTime macht die Unsicherheit **explizit und garantiert beschränkt**, anstatt sie zu ignorieren. Dadurch kann Spanner globale Transaktionen mit externer Konsistenz ohne zentralen Lock-Manager durchführen.

## Aufgabe 3

### Kausale Abhängigkeit

Kausal abhängig:

$$a_1 \rightarrow a_2 \rightarrow a_3 \rightarrow a_4 \rightarrow a_5$$

$$b_1 \rightarrow b_2 \rightarrow b_3 \rightarrow b_4 \rightarrow b_5$$

$$c_1 \rightarrow c_2 \rightarrow c_3 \rightarrow c_4$$

$$a_1 \rightarrow c_1$$

$$b_1 \rightarrow a_2$$

$$b_2 \rightarrow c_2$$

$$c_3 \rightarrow a_3$$

$$c_3 \rightarrow a_4$$

$$c_4 \rightarrow b_4$$

$$a_5 \rightarrow b_5$$

Kausal unabhängig (nicht vollständig):

$$a_2 \rightarrow c_2$$

$$a_3 \rightarrow c_4$$

$$b_4 \rightarrow a_5$$

...

## Lamport-Zeitstempel

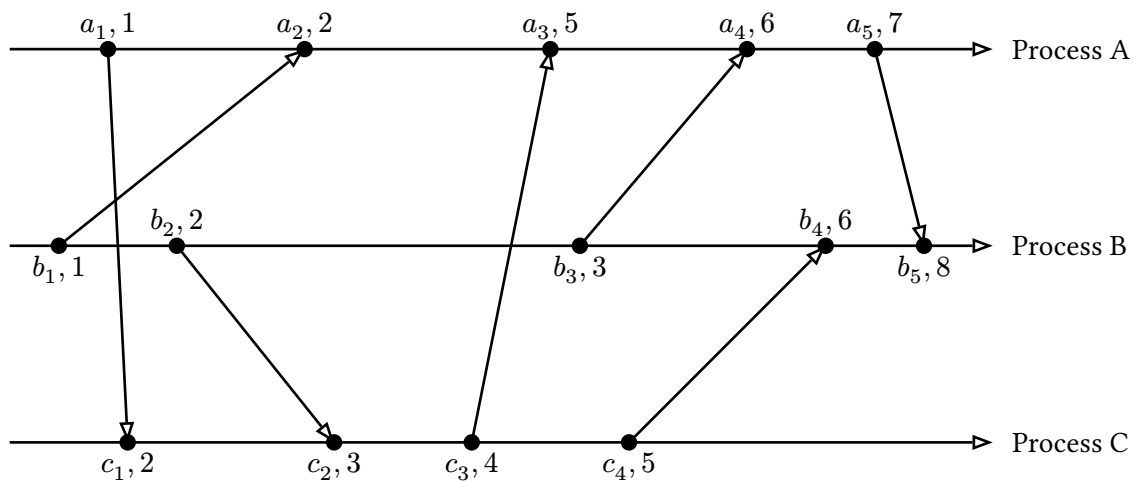


Abbildung 1: Logische Uhren in einem verteilten System, mit Lamport-Zeitstempeln

## Scalar Clock Condition

$$\forall e_1, e_2 \in E : e_1 \rightarrow e_2 \implies C(e_1) < C(e_2)$$

$$C(a_1) < C(c_1) \iff 1 < 2$$

$$C(b_1) < C(a_2) \iff 1 < 2$$

$$C(b_2) < C(c_3) \iff 2 < 3$$

$$C(c_3) < C(a_3) \iff 4 < 5$$

$$C(b_3) < C(a_4) \iff 3 < 6$$

$$C(c_4) < C(b_4) \iff 5 < 6$$

$$C(a_5) < C(b_5) \iff 7 < 8$$

$\implies$  Scalar Clock Condition ist erfüllt.

## Totale Ordnung

$$a_1 \rightarrow b_1 \rightarrow a_2 \rightarrow b_2 \rightarrow c_1 \rightarrow b_3 \rightarrow c_2 \rightarrow c_3 \rightarrow a_3 \rightarrow c_4 \rightarrow a_4 \rightarrow b_4 \rightarrow a_5 \rightarrow b_5$$

Die totale Ordnung kommt nach Anleitung der Vorlesungsfolien 46 - 47 zustande. Dabei wird angenommen, dass die Ordnung zwischen den Prozessen  $A < B < C$  ist .

## Vector Clock

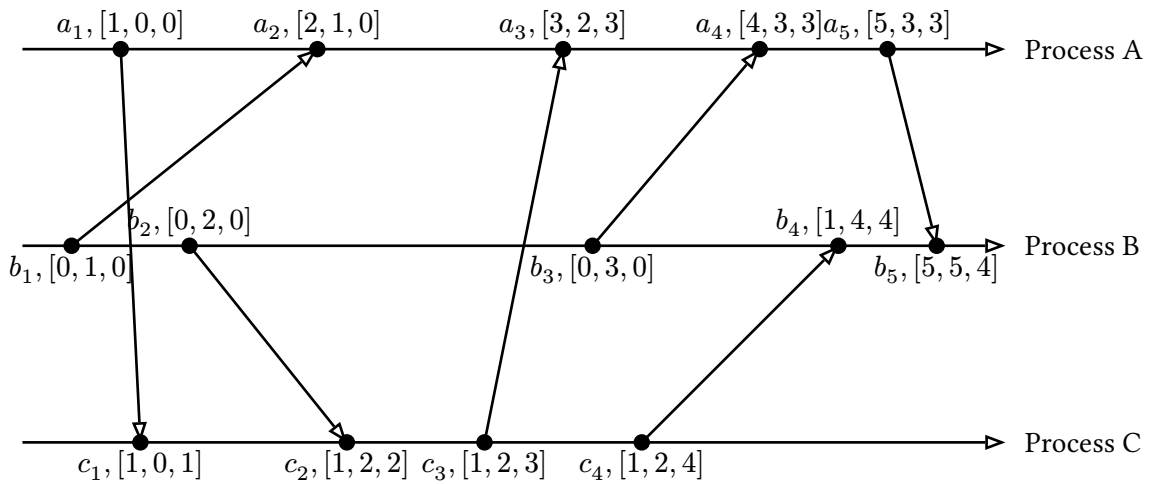


Abbildung 2: Logische Uhren in einem verteilten System, mit Vektor-Zeitstempeln

## Vector Clock Condition

$$\forall e_1, e_2 \in E : e_1 \rightarrow e_2 \implies V(e_1) < V(e_2)$$

Mit Vektorvergleichsoperation ( $<$ ):

$$\forall i : V(e_1)[i] \leq V(e_2)[i] \wedge \exists j : V(e_1)[j] < V(e_2)[j]$$

$$C(a_1) < C(c_1) \iff [1, 0, 0] < [1, 0, 1]$$

$$C(b_1) < C(a_2) \iff [0, 1, 0] < [2, 1, 0]$$

$$C(b_2) < C(c_3) \iff [0, 2, 0] < [1, 2, 2]$$

$$C(c_3) < C(a_3) \iff [1, 2, 3] < [3, 2, 3]$$

$$C(b_3) < C(a_4) \iff [0, 3, 0] < [4, 3, 3]$$

$$C(c_4) < C(b_4) \iff [1, 2, 3] < [1, 4, 4]$$

$$C(a_5) < C(b_5) \iff [5, 3, 3] < [5, 5, 4]$$

$\implies$  Vector Clock Condition ist erfüllt.

# Aufgabe 4

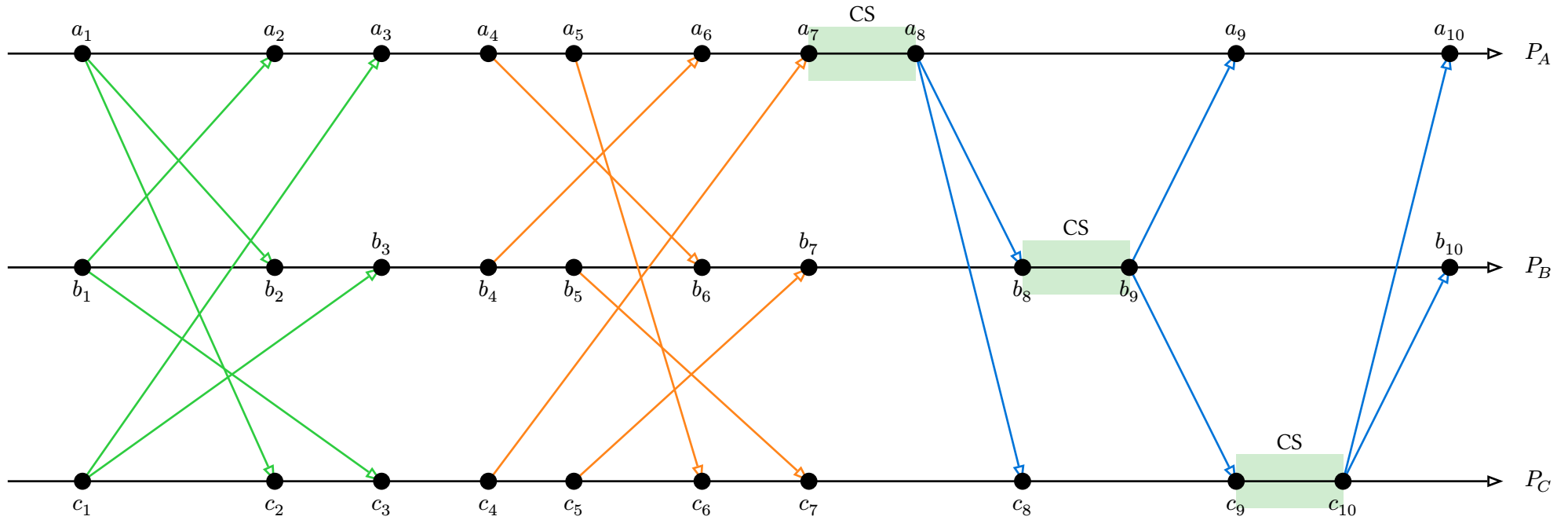


Abbildung 3: Lamport's Mutual Exclusion mit drei Prozessen ( $P_1 < P_2 < P_3$  bei Gleichstand)

$$\forall e_i \in E : C(e_i) = i$$

Grüne Pfeile repräsentieren Requests, orange Pfeile Acknowledgements, blaue Pfeile Releases.

Relevante Warteschlangen-Veränderungen:

$$\begin{aligned}
 a_1 &: [(A, 1)] \\
 b_1 &: [(B, 1)] \\
 c_1 &: [(C, 1)] \\
 a_2 &: [(A, 1), (B, 1)] \\
 b_2 &: [(A, 1), (B, 1)] \\
 c_2 &: [(A, 1), (C, 1)] \\
 a_3 = b_3 = c_3 &: [(A, 1), (B, 1), (C, 1)] \\
 a_8 = b_8 = c_8 &: [(B, 1), (C, 1)] \\
 a_9 = b_9 = c_9 &: [(C, 1)] \\
 a_{10} = b_{10} = c_{10} &: []
 \end{aligned}$$

### Delta-Kompression

Hier kann klassisches „Delta-Encoding“ verwendet werden. Dazu wird die element-weise Differenz zwischen zwei Vektoren gebildet:

$$\begin{aligned}
 \delta : \mathbb{Z}^n \times \mathbb{Z}^n &\rightarrow \mathbb{Z}^n \mid n \in \mathbb{N}^+ \\
 \delta(\vec{v}_a, \vec{v}_b) &= \vec{v}_b - \vec{v}_a
 \end{aligned}$$

Aus einem Anfangsvektor und einer Sequenz von beliebig vielen Delta-Vektoren kann jedes Zwischenergebnis sukzessive berechnet werden. Es geht also keine Information verloren:

$$\begin{aligned}
 \vec{v}_b &= \vec{v}_a + \delta(\vec{v}_a, \vec{v}_b) \\
 \vec{V} &= [\vec{v}_0, \vec{v}_1, \dots, \vec{v}_n] \\
 \vec{D} &= [\delta(\vec{v}_0, \vec{v}_1), \delta(\vec{v}_1, \vec{v}_2), \dots, \delta(\vec{v}_{n-1}, \vec{v}_n)] \\
 \vec{v}_i &= \vec{v}_0 + \sum_{j=1}^i \vec{D}_j
 \end{aligned}$$

An einem Beispiel:

$$\vec{v}_1 = \begin{pmatrix} 100000 \\ 200000 \\ 300000 \end{pmatrix} \quad \vec{v}_2 = \begin{pmatrix} 100005 \\ 200005 \\ 300005 \end{pmatrix} \quad \vec{v}_d = \delta(\vec{v}_1, \vec{v}_2) = \begin{pmatrix} 5 \\ 5 \\ 5 \end{pmatrix}$$

Liegen die Eingabevektoren nahe beieinander, haben die daraus resultierenden Vektoren deutlich kleinere Werte. Das Speichern/Verschicken der Delta-Vektoren benötigt also deutlich weniger Bits als die absoluten. Es wird nur ein einziger kompletter Vektor benötigt. Dieses Konzept ist bei verteilten Zeitsystemen sehr sinnvoll, da davon ausgegangen wird, dass der Clock Drift zwischen Teilnehmern nicht gigantisch ist. Daher liegen die Vektoren im Schnitt recht nah beieinander. Unter der Annahme, dass vorzeichenlose Ganzzahlen (z.B. Unix-Zeitstempel) kodiert werden und die Zeitstempel monoton steigend sind (also auch die Deltavektoren immer positiv sind) verbraucht der Deltavektor deutlich weniger Platz bei der Übertragung:

$$\text{bitlen}(n) := \begin{cases} 1 & | \ n = 0 \\ \lceil \log_2 n \rceil & | \ n \in \mathbb{N}^+ \end{cases}$$

$$\text{bitlen}(\vec{v}) := \sum_{i=1}^n \text{bitlen}(v_i) \mid \vec{v} \in \mathbb{N}^n$$

$$\text{bitlen}(\vec{v}_2) = 54$$

$$\text{bitlen}(\vec{v}_d) = 9$$